

1. What is a Class in Java?

A **class** is the **basic building block of Java**.

- A class **defines the structure of an object**.
- It describes **what data the object has** and **what actions it can perform**.

Think of it like a **blueprint**.

Example analogy 🏠

- **Class** → Blueprint of a house
- **Object** → Actual house built from the blueprint

So:

- **Class = Template**
- **Object = Instance of that template**

2. Class Creates a New Data Type

When you create a class, you are actually **creating a new data type**.

Example:

```
class Box {  
    double width;  
    double height;  
    double depth;  
}
```

Here **Box** becomes a **new data type**.

3. Members of a Class

Inside a class there are two main components:

Instance Variables (Data)

Variables that store object data.

Example:

```
double width;  
double height;  
double depth;
```

Each object has **its own copy** of these variables.

Methods (Functions)

Methods define **actions performed on data**.

Example:

```
void volume() {  
    System.out.println(width * height * depth);  
}
```

4. General Form of a Class

Basic structure:

```
class ClassName {  
  
    // instance variables  
    type variable1;  
    type variable2;  
  
    // methods  
    returnType methodName() {  
        // code  
    }  
}
```

5. Creating an Object

A class itself **does not create objects**.

To create an object, use the **new keyword**.

Example:

```
Box mybox = new Box();
```

Explanation:

Part	Meaning
Box	class type
mybox	object name
new Box()	creates object

6. Accessing Variables (Dot Operator)

To access variables inside an object we use the **dot operator (.)**

Example:

```
mybox.width = 100;
mybox.height = 20;
mybox.depth = 15;
```

Here:

- mybox.width means **width inside the mybox object**

7. Example Program

```
class Box {
    double width;
    double height;
    double depth;
}

class BoxDemo {
    public static void main(String args[]) {

        Box mybox = new Box();

        mybox.width = 10;
        mybox.height = 20;
        mybox.depth = 15;

        double volume;
        volume = mybox.width * mybox.height * mybox.depth;

        System.out.println("Volume is " + volume);
    }
}
```

Output:

Volume is 3000.0

8. Multiple Objects Have Separate Data

Example:

```
Box box1 = new Box();  
Box box2 = new Box();
```

Each object has **its own copy of variables**.

Example:

```
box1.width = 10;  
box2.width = 50;
```

These values are **independent**.

Object	width
box1	10
box2	50

Changing box1 **does NOT affect** box2.

Declaring Objects in Java

When you create a **class**, you create a **new data type**. But to use it, you must **create objects of that class**.

Creating an object happens in **two steps**.

1. Step 1: Declare a Reference Variable

First, you declare a variable of the class type.

Example:

```
Box mybox;
```

Explanation:

Part	Meaning
Box	class type
mybox	reference variable

Important:

- This **does NOT create an object**.
- It only creates a **reference variable** that can point to an object.

Think of it like a **remote control without a TV connected yet**.

2. Step 2: Create the Object using new

Now we create the actual object.

mybox = new Box();

Explanation:

Part	Meaning
new	allocates memory
Box()	constructor
mybox	stores the reference

Now:

- Memory is allocated
- Object is created
- Reference stored in mybox

Combined Form

Most Java programs combine both steps:

Box mybox = new Box();

This means:

1. Declare reference variable

2. Create object
3. Assign reference

All in one line.

How Memory Works

mybox -----> Box Object in Memory
 width
 height
 depth

mybox does **not contain the object itself**. It contains a **reference (memory address)** to the object.

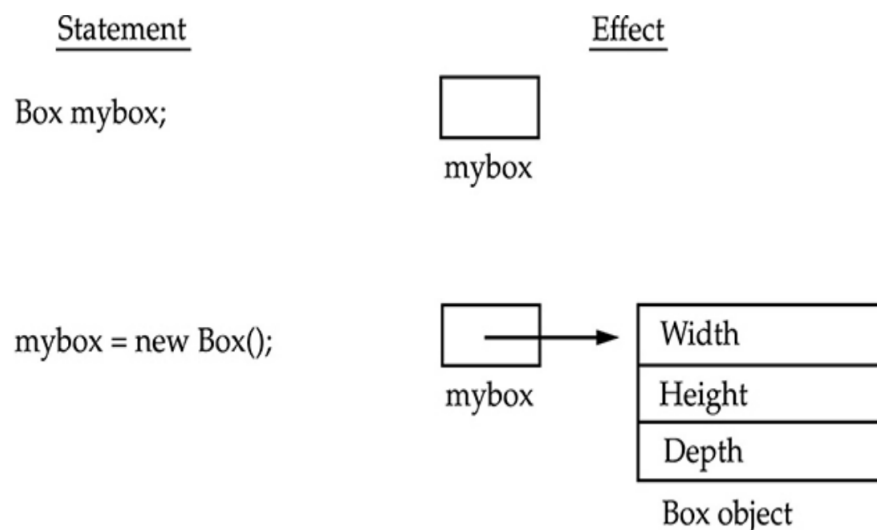


Figure 6-1 Declaring an object of type **Box**

Understanding the new Operator

General form: **classVariable = new ClassName();**

Example: **Box mybox = new Box();**

What new does:

1. Allocates memory at **runtime**
2. Creates the object
3. Calls the **constructor**
4. Returns the reference

Assigning Object Reference Variables

In Java, **object variables store references (addresses), not the actual object.**

Example:

```
Box b1 = new Box();  
Box b2 = b1;
```

What many beginners think happens ✕

They think Java creates **two separate objects**.

b1 ----> Box Object 1

b2 ----> Box Object 2

But this is **incorrect**.

What actually happens ✓

Both variables refer to **the same object**.

```
b1 ----\  
          ---> Box Object  
b2 ----/
```

So:

- **No new object is created**
- **No data is copied**
- **Only the reference is copied**

Example Program

```
class Box {  
    double width;  
}
```

```
class Test {  
    public static void main(String args[]) {  
  
        Box b1 = new Box();
```

```
Box b2 = b1;

b1.width = 10;

System.out.println(b2.width);
}
```

Output:

10

Why?

Because **b1 and b2 point to the same object.**

Changing Data Through One Reference

```
b2.width = 50;
```

Now the value becomes:

```
b1.width = 50
b2.width = 50
```

Because they refer to the **same object.**

Setting a Reference to null

Example:

```
b1 = null;
```

Memory structure now:

```
b1 ----> null
```

```
b2 ----> Box Object
```

Important points:

- The object **still exists**
- b2 still points to it
- Only b1 lost the reference

Very Important Rule

When you assign one object reference to another:

- ✓ The object is **NOT** copied
- ✓ Only the **reference** is copied

Example:

Box b2 = b1;

This copies the **reference**, not the object.

Introducing Methods

Classes usually contain:

Component	Purpose
Instance Variables	Store data
Methods	Perform operations

Methods define **what an object can do**.

General Form of a Method

```
returnType methodName(parameter-list) {  
  
    // method body  
  
}
```

Example:

```
int add(int a, int b) {  
    return a + b;  
}
```

Parts of a Method

1. Return Type

Specifies what value the method returns.

Examples:

int
double
String
boolean
void

Example:

int getNumber()

If the method **returns nothing**, use:

void

Example:

void display()

2. Method Name

The name used to call the method.

Example:

calculateVolume()

3. Parameters

Parameters receive values from the method call.

Example:

int add(int x, int y)

Here:

x and y → parameters

Example Method

```
class Box {  
  
    double width;  
    double height;  
    double depth;  
  
    double volume() {  
        return width * height * depth;  
    }  
}
```

Calling the Method

```
Box mybox = new Box();
```

```
mybox.width = 10;  
mybox.height = 5;  
mybox.depth = 2;
```

```
double v = mybox.volume();
```

```
System.out.println(v);
```

Output:

100

Return Statement

If a method returns a value, use:

return value;

Example:

return width * height * depth;

This sends the result **back to the calling code**.

Example Without Return Value

```
void display() {  
    System.out.println("Hello");  
}
```

Return type = void

Meaning:

This method returns nothing

```
class Box {
    double width;
    double height;
    double depth;

    // compute and return volume
    double volume() {
        return width * height * depth;
    }
}

class BoxDemo4 {
    public static void main(String[] args) {
        Box mybox1 = new Box();
        Box mybox2 = new Box();
        double vol;

        // assign values to mybox1's instance variables
        mybox1.width = 10;
        mybox1.height = 20;
        mybox1.depth = 15;

        /* assign different values to mybox2's
           instance variables */
        mybox2.width = 3;
        mybox2.height = 6;
        mybox2.depth = 9;

        // get volume of first box
        vol = mybox1.volume();
        System.out.println("Volume is " + vol);

        // get volume of second box
        vol = mybox2.volume();
        System.out.println("Volume is " + vol);
    }
}
```

Adding a Method That Takes Parameters

Some methods work without parameters, but most methods **need input values** to work with.

Parameters make methods **more flexible and reusable**.

Example Without Parameters

```
int square() {  
    return 10 * 10;  
}
```

Output will **always be 100**. So this method is **not useful** because it only squares **10**.

Example With Parameters

```
int square(int i) {  
    return i * i;  
}
```

Now the method can calculate **any square value**.

Example calls:

```
square(5); // 25  
square(9); // 81  
square(2); // 4
```

Now the method is **general-purpose**.

Parameter vs Argument

This is an **important concept in Java**.

Term	Meaning
Parameter	Variable in method definition
Argument	Value passed when method is called

Example

```
int square(int i)
```

Here:

$i \rightarrow$ parameter

Method call:

```
square(100);
```

Here:

100 → argument

Inside the method:

```
i = 100
```

Improving the Box Class

Earlier, box dimensions were set like this:

```
mybox.width = 10;  
mybox.height = 20;  
mybox.depth = 15;
```

Problems:

- ✗ Too many lines
- ✗ Easy to forget a value
- ✗ Breaks good OOP practice

Better Approach: Use a Method

Create a method to set dimensions.

```
void setDim(double w, double h, double d) {  
    width = w;  
    height = h;  
    depth = d;  
}
```

Complete Example

```
class Box {  
  
    double width;  
    double height;  
    double depth;  
  
    void setDim(double w, double h, double d) {  
        width = w;
```

```

        height = h;
        depth = d;
    }

    double volume() {
        return width * height * depth;
    }
}

class BoxDemo {

    public static void main(String args[]) {

        Box mybox1 = new Box();
        Box mybox2 = new Box();

        mybox1.setDim(10, 20, 15);
        mybox2.setDim(3, 6, 9);

        System.out.println("Volume1 = " + mybox1.volume());
        System.out.println("Volume2 = " + mybox2.volume());
    }
}

```

How the Method Call Works

Example:

mybox1.setDim(10, 20, 15);

Values are copied like this:

Argument	Parameter
10	w
20	h
15	d

Then inside the method:

```
width = w;  
height = h;  
depth = d;
```

Constructors in Java

A **constructor** is a special method that **automatically runs when an object is created**.

Its main purpose is to **initialize the object**.

Example:

```
class Box {  
    double width;  
    double height;  
    double depth;  
  
    Box() {  
        width = 10;  
        height = 10;  
        depth = 10;  
    }  
}
```

Now when we create an object:

```
Box mybox = new Box();
```

The constructor **Box()** is **automatically called**.

Important Properties of Constructors

- ✓ Constructor name = **same as class name**
- ✓ **No return type** (not even void)
- ✓ Automatically called when object is created
- ✓ Used to **initialize variables**

Example Program

```
class Box {
```

```

double width;
double height;
double depth;

Box() {
    width = 10;
    height = 10;
    depth = 10;
}

double volume() {
    return width * height * depth;
}
}

class BoxDemo {

    public static void main(String args[]) {

        Box mybox1 = new Box();
        Box mybox2 = new Box();

        System.out.println(mybox1.volume());
        System.out.println(mybox2.volume());
    }
}

```

Output:

```

1000
1000

```

Because both boxes were initialized to:

$10 \times 10 \times 10$

Why Constructors Are Useful

Without constructors you would need:

```
Box mybox = new Box();
```

```
mybox.width = 10;
```

```
mybox.height = 10;  
mybox.depth = 10;
```

Constructors make code **shorter and safer**.

Types of Constructor:

1. Default Constructor- A constructor with no **parameters**.

```
class Box {  
    double width;  
    double height;  
    double depth;  
Box()  
{  
width=0.0;  
height=0.0;  
depth=0.0;  
}  
  
}
```

If you **do not write a constructor**, Java automatically creates one.

Example:

```
class Box {  
    double width;  
    double height;  
    double depth;  
}
```

Java creates a **default constructor**.

Default values become:

Type	Default Value
numeric	0
boolean	false
object reference	null

2. Parameterized Constructors

A constructor can also accept **parameters**.

This allows different objects to have **different values**.

Example:

```
class Box {  
  
    double width;  
    double height;  
    double depth;  
  
    Box(double w, double h, double d) {  
        width = w;  
        height = h;  
        depth = d;  
    }  
  
    double volume() {  
        return width * height * depth;  
    }  
}
```

Creating Objects with Different Values

```
Box mybox1 = new Box(10, 20, 15);
```

```
Box mybox2 = new Box(3, 6, 9);
```

Now:

Object	width	height	depth
mybox1	10	20	15
mybox2	3	6	9

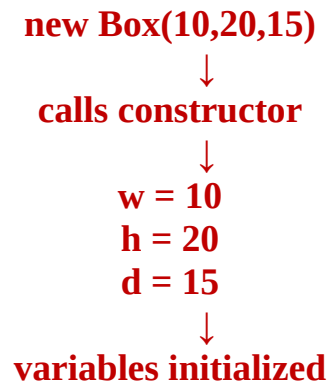
So each object gets **different dimensions**.

How Constructor Is Called

Example:

```
Box mybox1 = new Box(10,20,15);
```

Flow:



The this Keyword

this refers to the **current object**.

Example:

this.width

means , **width of current object**

Example Using this

```
class Box {  
  
    double width;  
    double height;  
    double depth;  
  
    Box(double w, double h, double d) {  
        this.width = w;  
        this.height = h;  
        this.depth = d;  
    }  
}
```

Here:

Code	Meaning
this.width	instance variable
w	parameter

Instance Variable Hiding

Sometimes **parameter names and instance variable names are the same.**

Example:

```
class Box {  
  
    double width;  
    double height;  
    double depth;  
  
    Box(double width, double height, double depth) {  
  
        this.width = width;  
        this.height = height;  
        this.depth = depth;  
    }  
}
```

Here:

Variable	Meaning
width (right side)	parameter
this.width	instance variable

So:

this.width = width

means

instanceVariable = parameter

Why this keyword Is Needed

Without this:

width = width;

Java thinks both refer to the **parameter**, so the instance variable remains unchanged.

Using this solves this problem.

Example

Constructor Call

new Box(10,20,15)

Parameters:

width = 10

height = 20

depth = 15

Instance Variables:

this.width = 10

this.height = 20

this.depth = 15

Example:

Java Example: Default + Parameterized Constructor

```
class Box {
```

```
    double width;
```

```
    double height;
```

```
    double depth;
```

```
    // Default Constructor
```

```
    Box() {
```

```
        width = 10;
```

```
        height = 10;
```

```
        depth = 10;
```

```
        System.out.println("Default Constructor Called");
```

```
    }
```

```
    // Parameterized Constructor
```

```
    Box(double w, double h, double d) {
```

```
        width = w;
```

```
        height = h;
```

```
        depth = d;
```

```
        System.out.println("Parameterized Constructor Called");
```

```
    }
```

```
    double volume() {
```

```
        return width * height * depth;
    }
}

class BoxDemo {

    public static void main(String args[]) {

        // Using default constructor
        Box box1 = new Box();

        // Using parameterized constructor
        Box box2 = new Box(5, 6, 7);

        System.out.println("Volume of box1 = " + box1.volume());
        System.out.println("Volume of box2 = " + box2.volume());
    }
}
```

Output

Default Constructor Called
Parameterized Constructor Called
Volume of box1 = 1000.0
Volume of box2 = 210.0

How This Program Works

1. Default Constructor

Box box1 = new Box();

Calls:

Box()

Values assigned:

width = 10
height = 10
depth = 10

Volume:

$$10 \times 10 \times 10 = 1000$$

2. Parameterized Constructor

Box box2 = new Box(5,6,7);

Calls:

Box(double w, double h, double d)

Values assigned:

width = 5

height = 6

depth = 7

Volume:

$$5 \times 6 \times 7 = 210$$

Method Overloading

Method Overloading means defining **multiple methods with the same name** in the same class but with **different parameter lists**.

Java determines which method to call based on:

- **Number of parameters**
- **Type of parameters**

Return type alone cannot distinguish overloaded methods.

Example of Method Overloading

```
class OverloadDemo {  
  
    void test() {  
        System.out.println("No parameters");  
    }  
  
    void test(int a) {  
        System.out.println("One integer: " + a);  
    }  
  
    void test(int a, int b) {  
        System.out.println("Two integers: " + a + " " + b);  
    }  
  
    double test(double a) {  
        System.out.println("One double: " + a);  
        return a * a;  
    }  
}  
  
class TestOverload {  
  
    public static void main(String args[]) {  
  
        OverloadDemo obj = new OverloadDemo();  
  
        obj.test();  
        obj.test(10);  
        obj.test(10, 20);  
    }  
}
```

```
        double result = obj.test(5.5);
        System.out.println("Square = " + result);
    }
}
```

Output

No parameters
One integer: 10
Two integers: 10 20
One double: 5.5
Square = 30.25

Important Rules of Method Overloading

Valid Overloading:

```
test()
test(int a)
test(int a, int b)
test(double a)
```

Invalid Overloading:

```
int test(int a)
double test(int a)
```

Because **only return type changed**, parameters are the same.

Automatic Type Conversion in Overloading

Java may **automatically convert types** if an exact match is not found.

Example:

```
class Demo {

    void test(double a) {
        System.out.println("Double version called: " + a);
    }

    public static void main(String args[]) {
```

```
Demo obj = new Demo();  
  
    obj.test(10); // int passed  
}  
}
```

Here:

int → **converted to** → **double** So test(double) is called.

Why Method Overloading is Useful

Without overloading, different method names are required.

Example in **C language**:

```
abs() → integer  
labs() → long  
fabs() → float
```

But in Java we can use **one name**:

```
abs(int)  
abs(long)  
abs(double)
```

This follows the idea:

One Interface → Multiple Methods

This is a form of **polymorphism**.

Constructor Overloading

Constructors can also be overloaded.

Example:

```
class Box {  
  
    double width;  
    double height;  
    double depth;  
  
    // constructor with 3 parameters
```

```

Box(double w, double h, double d) {
    width = w;
    height = h;
    depth = d;
}

// default constructor
Box() {
    width = -1;
    height = -1;
    depth = -1;
}

// cube constructor
Box(double len) {
    width = height = depth = len;
}

double volume() {
    return width * height * depth;
}
}

class OverloadCons {

    public static void main(String args[]) {

        Box box1 = new Box(10,20,15);
        Box box2 = new Box();
        Box cube = new Box(7);

        System.out.println("Volume box1 = " + box1.volume());
        System.out.println("Volume box2 = " + box2.volume());
        System.out.println("Volume cube = " + cube.volume());
    }
}

```

Output

```

Volume box1 = 3000
Volume box2 = -1
Volume cube = 343

```

Types of Constructors in this Example

Constructor	Purpose
Box()	default box
Box(double w,h,d)	box with dimensions
Box(double len)	cube

Using Objects as Parameters

Objects can also be passed as parameters to methods.

Example:

```
class Test {  
  
    int a;  
  
    Test(int i) {  
        a = i;  
    }  
  
    boolean equalTo(Test o) {  
  
        if(o.a == a)  
            return true;  
        else  
            return false;  
    }  
}  
  
class ObjectParam {  
  
    public static void main(String args[]) {  
  
        Test obj1 = new Test(100);  
        Test obj2 = new Test(100);  
        Test obj3 = new Test(200);  
  
        System.out.println(obj1.equalTo(obj2));  
        System.out.println(obj1.equalTo(obj3));  
    }  
}
```

```
}  
}
```

Output

true
false

Here:

equalTo(Test o)

The method receives **an object parameter**.

Copy Constructor (Object as Constructor Parameter)

Sometimes we want to **create an object using another object**.

Example:

```
class Box {  
  
    double width;  
    double height;  
    double depth;  
  
    // copy constructor  
    Box(Box ob) {  
        width = ob.width;  
        height = ob.height;  
        depth = ob.depth;  
    }  
  
    // normal constructor  
    Box(double w, double h, double d) {  
        width = w;  
        height = h;  
        depth = d;  
    }  
  
    double volume() {  
        return width * height * depth;  
    }  
}
```

```

class CopyDemo {

    public static void main(String args[]) {

        Box box1 = new Box(10,20,15);

        Box box2 = new Box(box1); // copy constructor

        System.out.println("Volume box1 = " + box1.volume());
        System.out.println("Volume box2 = " + box2.volume());
    }
}

```

Output

```

Volume box1 = 3000
Volume box2 = 3000

```

Now **box2** is initialized using **box1**.

Argument Passing in Java

There are **two ways** a programming language can pass arguments to a **method**:

1. Call-by-Value

- A **copy of the actual value** is passed to the method.
- Changes made inside the method **do not affect the original variable**.

Example (Primitive Types)

```

class Test {
    void meth(int a, int b) {
        a = 30;
        b = 10;
    }

    public static void main(String args[]) {
        Test obj = new Test();

        int x = 15;
        int y = 20;
    }
}

```

obj.meth(x, y);

```
        System.out.println("x = " + x);
        System.out.println("y = " + y);
    }
}
```

Output

x = 15
y = 20

✓ The values of **x and y do not change** because primitives are passed **by value**.

2. Passing Objects to Methods

In Java, **objects behave like call-by-reference**.

When an object is passed:

- The **reference of the object** is copied.
- Both references point to the **same object**.
- Changes inside the method **affect the original object**.

Example

```
class Test {
    int a, b;

    Test(int i, int j) {
        a = i;
        b = j;
    }

    void meth(Test obj) {
        obj.a = obj.a * 2;
        obj.b = obj.b / 2;
    }
}

class Demo {
    public static void main(String args[]) {
```

```

    Test ob = new Test(10, 20);
    System.out.println("Before: " + ob.a + " " + ob.b);

    ob.meth(ob);

    System.out.println("After: " + ob.a + " " + ob.b);
}
}

```

Output

Before: 10 20
After: 20 10

✓ The values change because **both references point to the same object**.

Important Note

Java **always uses call-by-value**, but:

Type	What is passed	Effect
Primitive	Copy of value	Original unchanged
Object	Copy of reference	Object can change

Returning Objects

A method in Java can **return an object** instead of primitive data.

Example

```

class Test {

    int a;

    Test(int i) {
        a = i;
    }

    Test incrByTen() {

```

```
        Test temp = new Test(a + 10);
        return temp;
    }
}

class Demo {
    public static void main(String args[]) {

        Test obj1 = new Test(10);
        Test obj2;

        obj2 = obj1.incrByTen();

        System.out.println("obj1.a = " + obj1.a);
        System.out.println("obj2.a = " + obj2.a);
    }
}
```

Output

```
obj1.a = 10
obj2.a = 20
```

✓ The method **creates a new object and returns it.**

Access Protection

Access Protection

- Java provides **access control** through **access specifiers**.
- These determine **which parts of the code** can access **members (variables/methods)** of a class.
- **Class**: Smallest unit of abstraction, holds **data (variables)** and **code (methods)**.
- **Package**: A collection of related classes. Helps organize and group classes.
- Both classes and packages define **scope** and **visibility**.

Access protection in Packages

4 access specifiers

- private-only inside class
- public- universal
- protected- same package level and only for subclass of different packages
- default- only in same package

Because of the interplay between classes and packages, Java addresses **four categories of visibility for class members:**

- **Subclasses in the same package**
- **Non-subclasses in the same package**
- **Subclasses in different packages**
- **Classes that are neither in the same package nor subclasses**

1. Subclasses in the same package
2. Non-subclasses in the same package
3. Subclasses in different packages
4. Non-subclasses in different packages

- Inherit and exist in the same package
- Just other classes in the same package
- Inherit but from a different package
- No relation or package similarity

The three access specifiers, private, public, and protected, provide a variety of ways to produce the many levels of access required by these categories.

	Private	No Modifier	Protected	Public
Same class	Yes	Yes	Yes	Yes
Same package subclass	No	Yes	Yes	Yes
Same package non-subclass	No	Yes	Yes	Yes
Different package subclass	No	No	Yes	Yes
Different package non-subclass	No	No	No	Yes

TABLE 9-1
Class Member
Access

	Private	No Modifier	Protected	Public
Same class	Yes	Yes	Yes	Yes
Same package subclass	No	Yes	Yes	Yes
Same package non-subclass	No	Yes	Yes	Yes
Different package subclass	No	No	Yes	Yes
Different package non-subclass	No	No	No	Yes

1. Same Class

Access: private, *default*, protected, public — All accessible

package pack1;

```
public class SameClass {
    private int a = 10;
    int b = 20;          // default
    protected int c = 30;
    public int d = 40;

    public void show() {
        System.out.println(a); //
        System.out.println(b); //
        System.out.println(c); //
        System.out.println(d); //
    }
}
```

2. Subclass in Same Package

Access: *default*, protected, public — private is NOT accessible

package pack1;

```
class SuperClass {
    private int a = 10;
    int b = 20;
    protected int c = 30;
    public int d = 40;
```

```

}

public class SubSamePackage extends SuperClass {
    public void display() {
        // System.out.println(a);    private not accessible
        System.out.println(b); //    default
        System.out.println(c); //    protected
        System.out.println(d); //    public
    }
}

```

3. Non-subclass in Same Package

Access: *default*, protected, public — private is NOT accessible

package pack1;

```

class AnotherClass {
    private int a = 10;
    int b = 20;
    protected int c = 30;
    public int d = 40;
}

public class NonSubSamePackage {
    public void test() {
        AnotherClass obj = new AnotherClass();
        // System.out.println(obj.a);    private
        System.out.println(obj.b); //    default
        System.out.println(obj.c); //    protected
        System.out.println(obj.d); //    public
    }
}

```

4. Subclass in Different Package

Access: protected, public — *default* and private are NOT accessible

// **File: SuperClass.java**

package pack1;

```

public class SuperClass {
    private int a = 10;
    int b = 20;
    protected int c = 30;
    public int d = 40;
}

```

```
}
```

// File: SubDiffPackage.java

```
package pack2;
```

```
import pack1.SuperClass;
```

```
public class SubDiffPackage extends SuperClass {  
    public void show() {  
        // System.out.println(a);    private  
        // System.out.println(b);    default (package-private)  
        System.out.println(c); //    protected  
        System.out.println(d); //    public  
    }  
}
```

5. Non-subclass in Different Package

Access: Only public — all others are NOT accessible

// File: TestClass.java

```
package pack1;
```

```
public class TestClass {  
    private int a = 10;  
    int b = 20;  
    protected int c = 30;  
    public int d = 40;  
}
```

// File: OutsideClass.java

```
package pack2;
```

```
import pack1.TestClass;
```

```
public class OutsideClass {  
    public void print() {  
        TestClass obj = new TestClass();  
        // System.out.println(obj.a);    private  
        // System.out.println(obj.b);    default  
        // System.out.println(obj.c);    protected — not subclass  
        System.out.println(obj.d); //    public  
    }  
}
```

Importing packages:

- Java uses packages to organize classes.
- To avoid writing full class names with their packages repeatedly, Java provides the import statement.
- Once imported, classes can be used by their simple names.

Example:

```
import java.util.Date; // Now you can use 'Date' directly
```

Syntax of import:

```
import packageName.ClassName; // To import a specific class
```

```
import packageName.*; // To import all public classes in a package
```

Examples:

```
import java.util.Scanner; // single class
```

```
import java.io.*; // all classes from java.io
```

- It appears **after package declaration** (if any) and **before class definitions**.

```
package myapp;
import java.util.Scanner;

public class MyClass {
    // code here
}
```

- Using * imports **all public classes** in a package, **but not sub-packages**.
- Better to import specific classes to avoid confusion and reduce compilation time.

Example:

```
import java.util.*; // Imports ArrayList, Scanner, etc.
```

Public Classes in Custom Packages

To access a class from another package:

- The class and its members must be declared public.
- It should be placed in a .java file with the **same name**.

// File: MyPack/Balance.java

```
package MyPack;
```

```
public class Balance {  
    String name;  
    double bal;  
  
    public Balance(String n, double b) {  
        name = n;  
        bal = b;  
    }  
  
    public void show() {  
        if(bal < 0)  
            System.out.println(name + "and" + bal);  
    }  
}
```

Using Imported Public Class from Another Package

// File: TestBalance.java

```
import MyPack.*;  
  
public class TestBalance {  
    public static void main(String[ ] args) {  
        Balance test = new Balance("J. J. Jaspers", 99.88);  
        test.show();  
    }  
}
```

This works because:

- Balance is declared public
- Its constructor and methods are also public

If Not Public — It Fails

If you remove public from Balance class, it becomes **package-private**, and:

// File in another package:

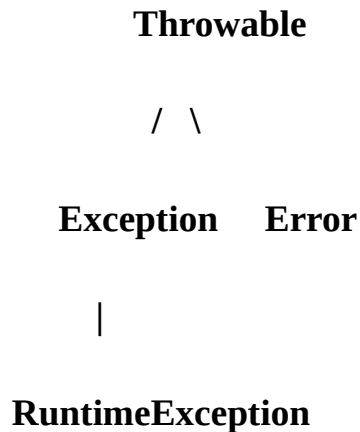
```
import MyPack.*;

public class TestBalance {
    public static void main(String[] args) {
        Balance b = new Balance("Name", 100.00); //    Compile-time error
    }
}
```

Exceptions:

In Java, exceptions are unexpected events that disrupt the normal flow of a program. All exceptions are objects that are derived from the built-in class `Throwable`.

Exception Class Hierarchy:



1. Throwable

The **base class** for anything that can be thrown using `throw`.

It has two direct subclasses:

- `Exception`
- `Error`

2. Exception

This class represents conditions that **a program should catch and handle**. You can also create your own custom exceptions by extending this class.

Common subclasses of Exception:

- `IOException`
- `SQLException`
- `ClassNotFoundException`

These are known as **checked exceptions**, which means the compiler forces you to handle them (either with `try-catch` or `throws`).

3. RuntimeException

A subclass of Exception, this represents **unchecked exceptions**, which are **not required** to be handled explicitly.

Common Runtime Exceptions:

- ArithmeticException (e.g., divide by zero)
- ArrayIndexOutOfBoundsException
- NullPointerException
- NumberFormatException

You **can** handle them, but the compiler won't force you to.

4. Error

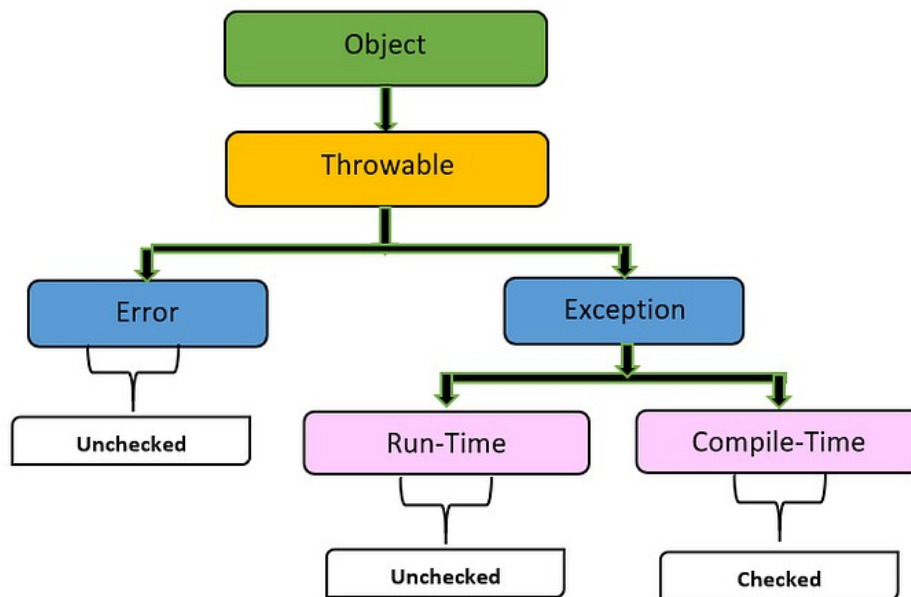
These are **serious problems** related to the **JVM or hardware**, and **should not be caught** by your program.

Common Errors:

- OutOfMemoryError
- StackOverflowError
- VirtualMachineError

Exception Types:

Checked and Unchecked Exceptions in Java



1. Checked Exceptions:

Checked exceptions, also known as compile-time exceptions, are exceptions that must be either caught or declared in the method signature using the `throws` keyword. These exceptions are typically used to handle expected error scenarios that a program can recover from. They force the developer to acknowledge and handle these exceptional conditions, ensuring that appropriate actions are taken.

Some common examples of checked exceptions in Java include:

- `IOException`: Thrown when an input or output operation fails, such as file I/O errors.
- `SQLException`: Thrown when there is an issue with database connectivity or queries.
- `FileNotFoundException`: Thrown when attempting to access a file that does not exist.
- `ClassNotFoundException`: Thrown when the Java runtime cannot find a specified class.

To handle a checked exception, you can use a **try-catch block**. This allows you to encapsulate the potentially problematic code within the try block and provide a set of instructions to handle the exception in the corresponding catch block. Here's an example:

```
try {
```

```

    // code that may throw a checked exception
}
catch (IOException e) {
    // handle the exception
}

```

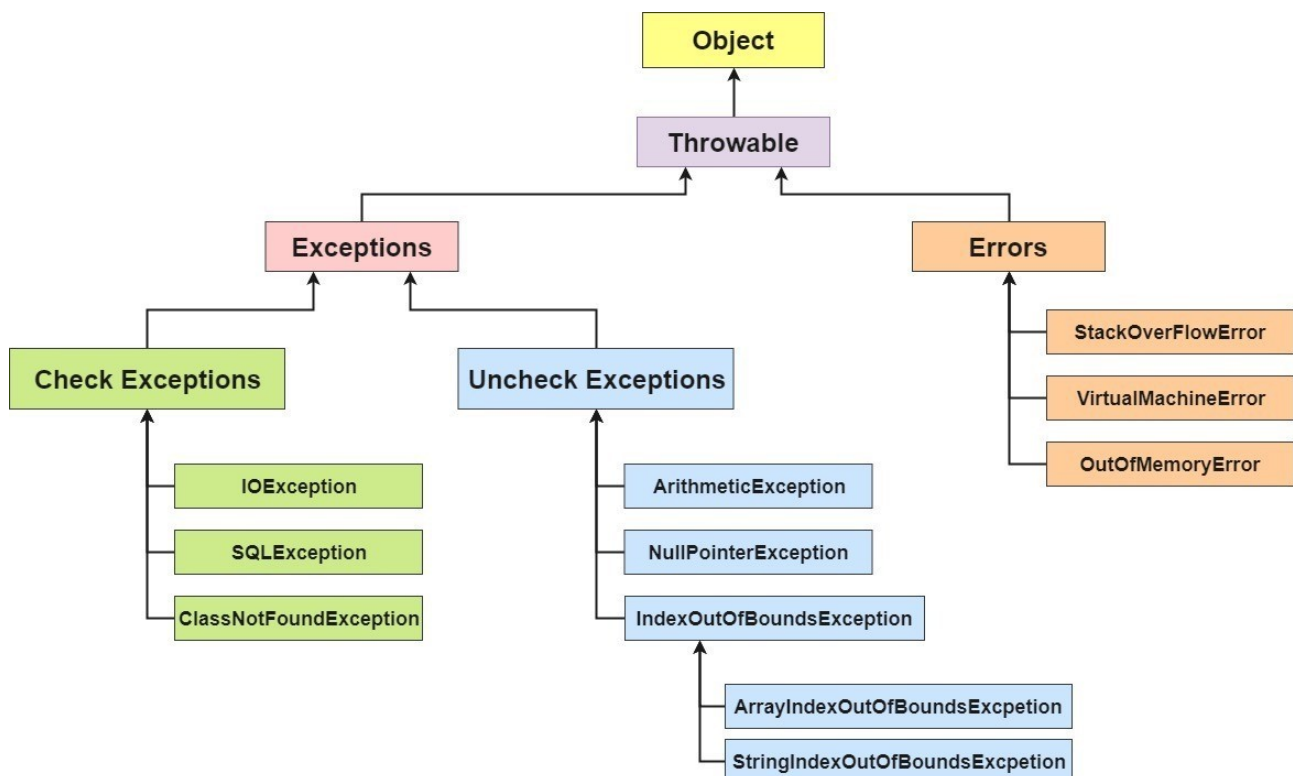
2. Unchecked Exceptions:

Unchecked exceptions, also known as runtime exceptions, are exceptions that do not need to be caught explicitly or declared using the throws keyword. They usually represent programming errors, such as invalid calculations or incorrect usage of APIs, that can be prevented by proper code design and testing.

Common examples of unchecked exceptions in Java include:

- `NullPointerException`: Thrown when an attempt is made to access a null object reference.
- `ArrayIndexOutOfBoundsException`: Thrown when trying to access an array element with an invalid index.
- `ArithmeticException`: Thrown when an arithmetic operation results in an error, such as division by zero.

Because unchecked exceptions typically indicate issues that should be fixed during development, they can often be avoided through proper coding practices, such as input validation and defensive programming.



Differences between Checked and Unchecked Exceptions in Java

S.No	Checked Exception	Unchecked Exception
1.	Checked exceptions happen at compile time when the source code is transformed into an executable code.	Unchecked exceptions happen at runtime when the executable program starts running.
2.	The checked exception is checked by the compiler.	These types of exceptions are not checked by the compiler.
3.	Checked exceptions can be created manually.	They can also be created manually.
4.	This exception is counted as a sub-class of the class.	This exception happens in runtime, and hence it is not included in the exception class.
5.	Java Virtual Machine requires the exception to be caught or handled.	Java Virtual Machine does not need the exception to be caught or handled.

throw and throws in Java

In Java, Exception Handling is one of the effective means to handle runtime errors so that the regular flow of the application can be preserved. It handles runtime errors such as **NullPointerException**, **ArrayIndexOutOfBoundsException**, etc. To handle these errors effectively, Java provides two key concepts **throw** and **throws**.

Difference Between throw and throws

The main **differences between throw and throws in Java** are follows:

Feature	throw	throws
Definition	It is used to explicitly throw an exception.	It is used to declare that a method might throw one or more exceptions.
Location	It is used inside a method or a block of code.	It is used in the method signature.
Usage	It can throw both checked and unchecked exceptions.	It is only used for checked exceptions. Unchecked exceptions do not require throws.
Responsibility	The method or block throws the exception.	The method's caller is responsible for handling the exception.

Feature	throw	throws
Flow of Execution	Stops the current flow of execution immediately.	It forces the caller to handle the declared exceptions.
Example	<pre>throw new ArithmeticException("Error") ;</pre>	<pre>public void myMethod() throws IOException {}</pre>

Java throw

The throw keyword in Java is used to explicitly throw an exception from a method or any block of code. We can throw either [checked or unchecked exception](#). The throw keyword is mainly used to throw custom exceptions.

Syntax of throw in Java

throw *Instance*

Where instance is an object of type Throwable (or its subclasses, such as Exception).

Example:

```
throw new ArithmeticException("/ by zero");
```

But this exception i.e., *Instance* must be of type **Throwable** or a subclass of **Throwable**.

The flow of execution of the program stops immediately after the throw statement is executed and the nearest enclosing **try** block is checked to see if it has a **catch** statement that matches the type of exception. If it finds a match, controlled is transferred to that statement otherwise next enclosing **try** block is checked, and so on. If no matching **catch** is found then the default exception handler will halt the program.

Example 1:

```
class Demo {
    static void fun()
    {
        try {
            throw new NullPointerException("demo");
        }
        catch (NullPointerException e) {
            System.out.println("Caught inside fun().");
            throw e;    // rethrowing the exception
        }
    }
}
```

```

    }
}

public static void main(String args[])
{
    try {
        fun();
    }
    catch (NullPointerException e) {
        System.out.println("Caught in main.");
    }
}
}

```

Output

Caught inside fun().
Caught in main.

Explanation: The above example demonstrates the use of the throw keyword to explicitly throw a [NullPointerException](#). The exception is caught inside the **fun()** method and rethrown, where it is then caught in the main() method.

Example 2:

```

class Demo {
    public static void main(String[] args)
    {
        System.out.println(1 / 0);
    }
}

```

Output:

Exception in thread "main" java.lang.ArithmeticException: / by zero

Java throws

throws is a keyword in Java that is used in the signature of a method to indicate that this method might throw one of the listed type exceptions. The caller to these methods has to handle the exception using a [try-catch block](#).

Syntax of Java throws

type method_name(parameters) throws exception_list

where, **exception_list** is a comma separated list of all the exceptions which a method might throw.

In a program, if there is a chance of raising an exception then the compiler always warns us about it and compulsorily we should handle that checked exception, Otherwise, we will get compile time error saying **unreported exception must be caught or declared to be thrown**. To prevent this compile time error we can handle the exception in two ways:

1. By using [try catch](#)
2. By using the **throws** keyword

We can use the throws keyword to delegate the responsibility of exception handling to the caller (It may be a method or JVM) then the caller method is responsible to handle that exception.

Explanation: In the above program, by using the throws keyword we handled the **InterruptedException** and we will get the output as **Hello Demo**.

Example : Throwing an Exception with throws

```
class Demo {  
  
    static void fun() throws IllegalAccessException  
    {  
        System.out.println("Inside fun(). ");  
        throw new IllegalAccessException("demo");  
    }  
  
    public static void main(String args[])  
    {  
        try {  
            fun();  
        }  
        catch (IllegalAccessException e) {  
            System.out.println("Caught in main.");  
        }  
    }  
}
```

Output

Inside fun().
Caught in main.

Important Points to Remember

- throws keyword is required only for checked exceptions and usage of the throws keyword for unchecked exceptions is meaningless.
- throws keyword is required only to convince the compiler and usage of the throws keyword does not prevent abnormal termination of the program.
- With the help of the throws keyword, we can provide information to the caller of the method about the exception.

finally Keyword in Java

The finally block in Java is **used to execute important code such as closing resources**, regardless of whether an exception is handled or not. It **always executes** after the try block and its corresponding catch blocks (if any), **even if an exception is not thrown** or a return statement is used.

Syntax:

```
try {  
    // code that may throw an exception  
} catch (ExceptionType e) {  
    // code to handle the exception  
} finally {  
    // code that will always execute  
}
```

- try block: contains code that might throw an exception.
- catch block: handles the exception.
- finally block: used for cleanup activities (closing files, releasing resources, etc.).

Example 1: Normal Case

```
public class FinallyDemo {  
    public static void main(String[] args) {  
        try {  
            System.out.println("Inside try block");  
        } catch (Exception e) {  
            System.out.println("Exception caught");  
        } finally {  
            System.out.println("Finally block executed");  
        }  
    }  
}
```

Output:

Inside try block
Finally block executed

Example 2: With Exception

```
public class FinallyDemo {  
    public static void main(String[] args) {  
        try {  
            int data = 50 / 0; // throws ArithmeticException  
        } catch (ArithmeticException e) {  
            System.out.println("Exception caught: " + e);  
        } finally {  
            System.out.println("Finally block executed");  
        }  
    }  
}
```

Output:

Exception caught: java.lang.ArithmeticException: / by zero
Finally block executed

Try with Multiple Catch Blocks

Syntax:

```
try {  
    // code that may throw multiple exceptions  
} catch (ExceptionType1 e1) {  
    // handle exception type 1  
} catch (ExceptionType2 e2) {  
    // handle exception type 2  
} catch (Exception e) {  
    // handle all other exceptions (optional but useful)  
}
```

Explanation:

You can catch **different types of exceptions separately** using multiple catch blocks. The most specific exceptions should come **first**, and the most general (like Exception) should come **last**.

Example:

```
public class MultipleCatchDemo {  
    public static void main(String[] args) {  
        try {  
            int a = 10 / 0;           // This will cause ArithmeticException  
            int[] arr = new int[5];  
        }  
    }  
}
```

```

        arr[10] = 100;          // This would cause ArrayIndexOutOfBoundsException
    } catch (ArithmeticException e) {
        System.out.println("Arithmetic Exception caught: " + e);
    } catch (ArrayIndexOutOfBoundsException e) {
        System.out.println("Array Index Exception caught: " + e);
    } catch (Exception e) {
        System.out.println("General Exception caught: " + e);
    }
}
}

```

Output:

Arithmetic Exception caught: java.lang.ArithmeticException: / by zero

Nested try Blocks

Syntax:

```

try {
    // Outer try block
    try {
        // Inner try block
    } catch (ExceptionType1 e1) {
        // Inner catch
    }
} catch (ExceptionType2 e2) {
    // Outer catch
}

```

Explanation:

- A try block inside another try block is called a **nested try**.
- Useful when different parts of the code can fail independently.

Example:

```

public class NestedTryDemo {
    public static void main(String[] args) {
        try {

```

```
System.out.println("Outer try block");

try {
    int num = 100 / 0; // ArithmeticException
} catch (ArithmeticException e) {
    System.out.println("Inner catch: " + e);
}

int[] arr = new int[3];
arr[5] = 50; // ArrayIndexOutOfBoundsException

} catch (ArrayIndexOutOfBoundsException e) {
    System.out.println("Outer catch: " + e);
}

System.out.println("Program continues...");
}
```

Output:

Outer try block

Inner catch: java.lang.ArithmeticException: / by zero

Outer catch: java.lang.ArrayIndexOutOfBoundsException: Index 5 out of bounds for length 3

Program continues...

Abstract Classes in Java

Abstract classes are used when we want to create a common base class that defines a generalized structure, but we want subclasses to fill in the details of certain methods. Abstract classes may have both **abstract methods** (methods without implementations) and **concrete methods** (methods with implementations). An abstract class cannot be instantiated directly, but it can serve as a base for other classes.

Let's break down the key concepts and then go through an example to demonstrate the use of abstract classes and methods.

Key Points:

1. **Abstract Method:** A method declared in an abstract class without a body. Subclasses are required to implement this method.

Syntax:

abstract returnType methodName(parameters);

- **abstract:** Keyword that defines the method as abstract.
- **returnType:** The type of the return value.
- **methodName:** The name of the method.
- **parameters:** The list of parameters (if any).

Example:

abstract void callme();

2. **Abstract Class:** A class that contains one or more abstract methods. It cannot be instantiated, but it can be subclassed.

abstract class ClassName {

// Abstract method (no implementation)

abstract void methodName();

// Concrete method (with implementation)

void concreteMethod() {

System.out.println("This is a concrete method.");

}

}

3. Concrete Class: A class that extends an abstract class and provides implementations for the abstract methods.

```
class Subclass extends AbstractClass {  
    void methodName( ) {  
        // Implement the abstract method  
    }  
}
```

Example:

Create the Abstract Class

In this example, we define an abstract class Figure with an abstract method area(). This method will be implemented by any subclass that extends Figure.

```
// Abstract class Figure  
abstract class Figure {  
    double dim1, dim2;  
  
    // Constructor to initialize dimensions  
    Figure(double a, double b) {  
        dim1 = a;  
        dim2 = b;  
    }  
  
    // Abstract method, no implementation  
    abstract double area();  
}
```

Create Concrete Subclasses

Now, we create two concrete subclasses: Rectangle and Triangle. Both classes override the abstract method area() to provide their own implementation.

```
// Rectangle class extending Figure  
class Rectangle extends Figure {  
    // Constructor to initialize dimensions  
    Rectangle(double a, double b) {  
        super(a, b);  
    }  
  
    // Overriding the area method for Rectangle  
    double area() {  
        System.out.println("Area of Rectangle:");  
    }  
}
```

```

        return dim1 * dim2;
    }
}

// Triangle class extending Figure
class Triangle extends Figure {
    // Constructor to initialize dimensions
    Triangle(double a, double b) {
        super(a, b);
    }

    // Overriding the area method for Triangle
    double area() {
        System.out.println("Area of Triangle:");
        return (dim1 * dim2) / 2;
    }
}

```

Use the Abstract Class and Subclasses

Now, we create a Main class to test our program. We use a reference of type Figure to point to objects of Rectangle and Triangle.

```

// Main class to test the Abstract Class and Subclasses
public class AbstractDemo {
    public static void main(String[] args) {
        // Creating objects of Rectangle and Triangle
        Figure figref;
        Rectangle r = new Rectangle(9, 5);
        Triangle t = new Triangle(10, 8);

        // Using Figure reference to point to Rectangle object
        figref = r;
        System.out.println("Area: " + figref.area());

        // Using Figure reference to point to Triangle object
        figref = t;
        System.out.println("Area: " + figref.area());
    }
}

```

Packages in Java

Package

A **package** is:

- A **namespace** that organizes classes and interfaces.
- A tool for **access control** — it can restrict visibility of classes or class members to within the package.

Think of packages like folders on your computer: they help you group related files together.

Why Use Packages?

Avoid name conflicts: Two developers can create a class with the same name (Foobar, Espresso, etc.), but if they're in different packages, it won't cause problems.

Better organization: You can logically group related classes together (e.g., UI-related classes in one package, database-related in another).

Access control: You can allow certain classes or methods to be accessed only within the same package.

Defining a Package

```
package MyPackage;
```

Now, every class in this file belongs to the **MyPackage** package.

Notes:

- If you **don't specify a package**, Java uses the **default package** (unnamed), which is fine for small programs, but not recommended for real applications.
- Package names and folder names must **match exactly** (case-sensitive).
 - `package myApp.utils;` → stored in `myApp/utils/` directory (on Windows/Linux).

Package Hierarchy

You can nest packages:

package com.mycompany.project.module;

- Folder structure: com/mycompany/project/module/
- Each . represents a new subdirectory.
- This is common in big projects or open-source libraries.

Multiple Files in a Package

Multiple .java files can be in the same package:

// In file: MyClass1.java

```
package MyPackage;  
public class MyClass1 {  
    // ...  
}
```

// In file: MyClass2.java

```
package MyPackage;  
public class MyClass2 {  
    // ...  
}
```

As long as both files declare the same package and are in the same folder, they're part of that package.

A Short Package Example

```
package MyPack;

class Balance {

    String name;

    double bal;

    Balance(String n, double b) {

        name = n;

        bal = b;

    }

}
```

```
void show() {

    if(bal<0)

    {

        System.out.println(name + "and" + bal);

    }

}
```

```
public class AccountBalance {  
    public static void main(String args[]) {  
        Balance current[ ] = new Balance[3];  
        current[0] = new Balance("K. J. Fielding", 123.23);  
        current[1] = new Balance("Will Tell", 157.02);  
        current[2] = new Balance("Tom Jackson", -12.33);  
        for (int i = 0; i < 3; i++) current[i].show();  
    }  
}
```

Compile the Code

Navigate to the parent directory of MyPack and run:

```
javac MyPack/AccountBalance.java
```

This compiles both Balance and AccountBalance classes into .class files inside the MyPack folder.

Run the Program

From the *parent* directory (i.e., the one that contains the MyPack folder), execute:

```
java MyPack.AccountBalance
```

You'll see this **output:**

K. J. Fielding: \$123.23

Will Tell: \$157.02

--> Tom Jackson: \$-12.33

Inheritance in Java

Inheritance is one of the core concepts in object-oriented programming (OOP) that allows a class to inherit properties and behaviors (methods) from another class. In Java, inheritance is implemented using the `extends` keyword. The class that is inherited from is called the **superclass**, and the class that inherits is called the **subclass**.

The basic idea of inheritance is to allow a subclass to inherit fields and methods from its superclass, while also having the ability to add new fields and methods or override the existing ones.

Syntax of Inheritance

The general syntax for inheritance in Java is as follows:

```
class Subclass extends Superclass {  
    // Additional fields and methods  
}
```

- **Subclass:** The class that inherits from another class.
- **Superclass:** The class that is being inherited.

Key Concepts

1. **Superclass:** The class from which other classes inherit. It can contain methods and fields that are inherited by its subclasses.
2. **Subclass:** The class that inherits the features (fields and methods) of another class.
3. **Accessing Superclass Members:** The subclass can access all the public and protected members of the superclass, but not private members (unless through a method).
4. **Overriding Methods:** A subclass can override methods of its superclass to provide its own implementation.
5. **Single Inheritance:** In Java, a class can inherit from only one superclass (Java does not support multiple inheritance).

Example:

```
// Superclass A  
class A {
```

```

int i, j; // Two integer variables

// Constructor to initialize i and j
void showij() {
    System.out.println("i and j: " + i + " " + j);
}
}

// Subclass B that inherits from class A
class B extends A {
    int k; // Additional variable for class B

    // Method to display k
    void showk() {
        System.out.println("k: " + k);
    }

    // Method to compute the sum of i, j, and k
    void sum() {
        System.out.println("i + j + k: " + (i + j + k));
    }
}

// Main class to test the inheritance
public class SimpleInheritance {
    public static void main(String args[]) {
        // Create an object of class A (superclass)
        A superOb = new A();
        superOb.i = 10; // Set i
        superOb.j = 20; // Set j
    }
}

```

```
System.out.println("Contents of superOb: ");  
superOb.showij(); // Call showij() of class A
```

```
System.out.println();
```

```
// Create an object of class B (subclass)
```

```
B subOb = new B();
```

```
subOb.i = 7; // Set i
```

```
subOb.j = 8; // Set j
```

```
subOb.k = 9; // Set k
```

```
System.out.println("Contents of subOb: ");
```

```
subOb.showij(); // Call showij() of class A (inherited by class B)
```

```
subOb.showk(); // Call showk() of class B
```

```
System.out.println();
```

```
System.out.println("Sum of i, j and k in subOb:");
```

```
subOb.sum(); // Call sum() of class B
```

```
}
```

```
}
```

In this example:

- Class B **extends** class A, meaning that class B inherits the fields i and j and the method showij() from class A.
- Class B also adds its own field k and two methods: showk() to display k and sum() to calculate the sum of i, j, and k.

Output:

Contents of superOb:

i and j: 10 20

Contents of subOb:

i and j: 7 8

k: 9

Sum of i, j and k in subOb:

$i + j + k$: 24

Interface

An **interface** in Java is a **reference type**, similar to a class, that can contain **abstract methods** (methods without a body), **default methods**, **static methods**, and **constants** (final variables).

- Interfaces are used to specify what a class **must do**, but **not how** it does it.
- Helps implement **polymorphism** (one interface, multiple methods).
- Provides a way to **achieve multiple inheritance** in Java.

Defining an Interface

Syntax:

```
interface InterfaceName {  
    // abstract methods (implicitly public and abstract)  
    returnType methodName1(parameterList);  
    returnType methodName2(parameterList);  
  
    // constants (implicitly public, static, final)  
    type CONSTANT_NAME = value;  
}
```

Example:

```
interface Callback {  
    void callback(int param);  
}
```

Implementing an Interface:

Syntax:

```
class ClassName implements InterfaceName {  
    // Implement all abstract methods from the interface  
    public returnType methodName1(parameterList) {  
        // method body  
    }  
}
```

Example:

```
class Client implements Callback {  
    public void callback(int p) {  
        System.out.println("callback called with " + p);  
    }  
}
```

Using Interface References

You can use an interface reference to call the method implemented by a class.

Example:

```
class TestIface {  
    public static void main(String[] args) {  
        Callback c = new Client(); // Interface reference to object  
        c.callback(42); // Output: callback called with 42  
    }  
}
```

You **cannot call** methods of the implementing class that are **not** in the interface.

Multiple Implementations

```
class AnotherClient implements Callback {  
    public void callback(int p) {  
        System.out.println("Another version of callback");  
        System.out.println("p squared is " + (p * p));  
    }  
}
```

```
class TestIface2 {  
    public static void main(String[] args) {  
        Callback c = new Client();    // First implementation  
        c.callback(42);                // Output: callback called with 42  
  
        c = new AnotherClient();      // Second implementation  
        c.callback(42);                // Output: Another version of callback, p squared is 1764  
    }  
}
```

Partial Implementation (Abstract Class)

If a class implements an interface but doesn't define all its methods, it must be declared **abstract**.

Example:

```
abstract class Incomplete implements Callback {
    int a, b;
    void show() {
        System.out.println(a + " " + b);
    }
    // callback() is not implemented, so this class is abstract
}
```

Nested Interfaces

An **interface inside a class or another interface** is called a **nested interface**.

Syntax:

```
class OuterClass {
    public interface NestedInterface {
        boolean isNotNegative(int x);
    }
}

class InnerClass implements OuterClass.NestedInterface {
    public boolean isNotNegative(int x) {
        return x >= 0;
    }
}
```

Usage Example:

```
class NestedIFDemo {
    public static void main(String[] args) {
        OuterClass.NestedInterface nif = new InnerClass();

        if (nif.isNotNegative(10))
            System.out.println("10 is not negative");
        if (nif.isNotNegative(-5))
            System.out.println("This won't be displayed");
    }
}
```

Variables in Interfaces:

```
interface SharedConstants {  
    int NO = 0;  
    int YES = 1;  
    int MAYBE = 2;  
    int LATER = 3;  
    int SOON = 4;  
    int NEVER = 5;  
}
```

Now use this in a class:

```
class Question implements SharedConstants {  
    Random rand = new Random();  
  
    int ask() {  
        int prob = (int) (100 * rand.nextDouble());  
        if (prob < 30) return NO;  
        else if (prob < 60) return YES;  
        else if (prob < 75) return LATER;  
        else if (prob < 98) return SOON;  
        else return NEVER;  
    }  
}
```

Extending Interfaces

One interface can **inherit from another** using `extends`.

```
interface A {  
    void meth1();  
    void meth2();  
}  
  
interface B extends A {  
    void meth3();  
}
```

Now, a class implementing **B** must implement **all three methods**:

```
class MyClass implements B {  
    public void meth1() {
```

```
        System.out.println("Implement meth1()");
    }
    public void meth2() {
        System.out.println("Implement meth2()");
    }
    public void meth3() {
        System.out.println("Implement meth3()");
    }
}
```

Example:

Create a simple Java program that demonstrates the use of interfaces. Define an interface `Vehicle` with two methods: `start()` and `stop()`. Then, create two classes, `Car` and `Bike`, that implement this interface. Each class should provide its own implementation for the `start()` and `stop()` methods. Finally, write a `Main` class to demonstrate how the `Car` and `Bike` objects can be referenced and invoked through the `Vehicle` interface.

// Vehicle.java

```
interface Vehicle {
    void start(); // Method to start the vehicle
    void stop(); // Method to stop the vehicle
}
```

// Car.java

```
class Car implements Vehicle {
    public void start() {
        System.out.println("Car is starting.");
    }

    public void stop() {
        System.out.println("Car is stopping.");
    }
}
```

// Bike.java

```
class Bike implements Vehicle {
    public void start() {
        System.out.println("Bike is starting.");
    }

    public void stop() {
        System.out.println("Bike is stopping.");
    }
}
```

```
}
```

// Main.java

```
public class Main {  
    public static void main(String[] args) {  
        // Create instances of Car and Bike  
        Vehicle myCar = new Car();  
        Vehicle myBike = new Bike();  
  
        // Call methods via the interface reference  
        myCar.start(); // Car is starting  
        myCar.stop(); // Car is stopping  
  
        myBike.start(); // Bike is starting  
        myBike.stop(); // Bike is stopping  
    }  
}
```

Output:

Car is starting.

Car is stopping.

Bike is starting.

Bike is stopping.

Understanding static in Java

In Java, the static keyword is used to indicate that a **field**, **method**, or **block** belongs to the class itself, rather than to instances (objects) of the class.

This means:

- A static **variable** is shared among all instances of a class.
- A static **method** can be called without creating an object.
- A static **block** is used to initialize static data and runs **once** when the class is loaded.

1) Java Static Variable

If we declare any variable as static, it is known as a static variable.

- The static variable can be used to refer to the common property of all objects (which is not unique for each object), for example, the company name of employees, college name of students, etc.
- The static variable gets memory only once in the class area at the time of class loading.
- Static variables in Java are also initialized to default values if not explicitly initialized by the programmer. They can be accessed directly using the class name without needing to create an instance of the class.
- Static variables are shared among all instances of the class, meaning if the value of a static variable is changed in one instance, it will reflect the change in all other instances as well.

Example of Static Variable

Example

```
//Java Program to demonstrate the use of static variable
class Student{
    int rollno;//instance variable
    String name;
    static String college ="ITS";//static variable
    //constructor
    Student(int r, String n){
        rollno = r;
        name = n;
    }
    //method to display the values
    void display () {System.out.println(rollno+" "+name+" "+college);}
```

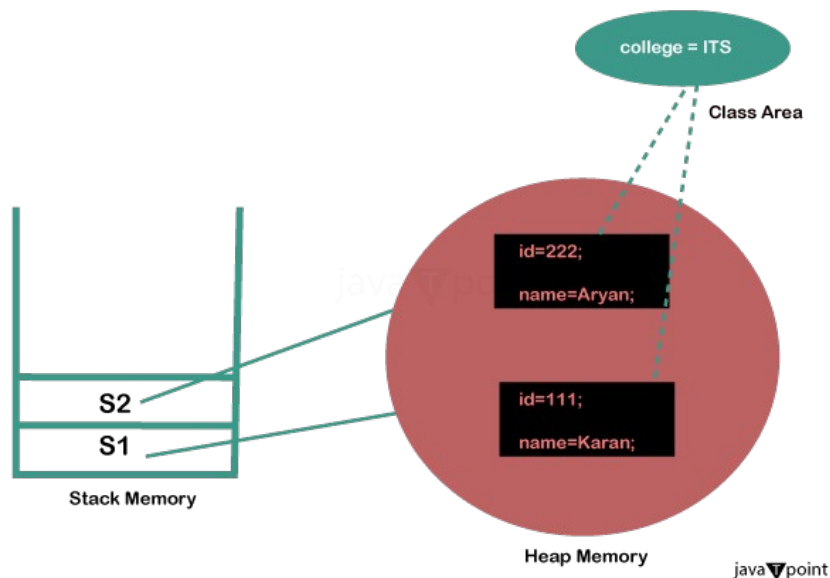
```

}
//Main class to show the values of objects
public class Main{
    public static void main(String args[]){
        Student s1 = new Student(111,"Karan");
        Student s2 = new Student(222,"Aryan");
        //we can change the college of all objects by the single line of code
        //Student.college="BBDIT";
        s1.display();
        s2.display();
    }
}

```

Output:

111 Karan ITS
222 Aryan ITS



Program of The Counter Without Static Variable

In this example, we have created an instance variable named `count`, which is incremented in the constructor. Since the instance variable gets the memory at the time of object creation, each object will have a copy of the instance variable. If it is incremented, it would not reflect other objects. So each object will have the value 1 in the `count` variable.

Example

```
//Java Program to demonstrate the use of an instance variable
//which get memory each time when we create an object of the class.
class Counter{
    int count=0;//will get memory each time when the instance is created

    Counter(){
        count++;//incrementing value
        System.out.println(count);
    }
}
public class Main{
    public static void main(String args[]){
        //Creating objects
        Counter c1=new Counter();
        Counter c2=new Counter();
        Counter c3=new Counter();
    }
}
```

Output:

```
1
1
1
```

Program of Counter by Static Variable

As we have mentioned above, a static variable will get the memory only once; if an object changes the value of the static variable, it will retain its value.

Example

```
//Java Program to illustrate the use of static variable which
//is shared with all objects.
class Counter{
    static int count=0;//will get memory only once and retains its value

    Counter(){
        count++;//incrementing value of static variable
        System.out.println(count);
    }
}
public class Main{
    public static void main(String args[]){
        //Creating objects
        Counter c1=new Counter();
        Counter c2=new Counter();
    }
}
```

```
        Counter c3=new Counter();
    }
}
```

Output:

```
1
2
3
```

2) Java Static Method

If we apply a static keyword with any method, it is known as a static method.

- A static method belongs to the class rather than the object of a class.
- A static method can be invoked without the need for creating an instance of a class.
- A static method can access static data members and can change their value of it.

Example

```
//Java Program to demonstrate the use of a static method.
class Student{
    int rollno;
    String name;
    static String college = "ITS";
    //static method to change the value of static variable
    static void change(){
        college = "BBDIT";
    }
    //constructor to initialize the variable
    Student(int r, String n){
        rollno = r;
        name = n;
    }
    //method to display values
    void display(){System.out.println(rollno+" "+name+" "+college);}
}
//Main class to create and display the values of object
public class Main{
    public static void main(String args[]){
        Student.change();//calling change method
        //creating objects
        Student s1 = new Student(111,"Karan");
        Student s2 = new Student(222,"Aryan");
    }
}
```

```

        Student s3 = new Student(333,"Sonoo");
        //calling display method
        s1.display();
        s2.display();
        s3.display();
    }
}

```

Output:

```

111 Karan BBDIT
222 Aryan BBDIT
333 Sonoo BBDIT

```

3) Java Static Block

- It is used to initialize the static data member.
- It is executed before the main() method at the time of class loading.

Example

```

//Java program to illustrate the use of static block
public class Main{
    static
    {
        System.out.println("static block is invoked");
    }
    public static void main(String args[]){
        System.out.println("main() method is invoked");
    }
}

```

Output:

```

static block is invoked
Hello main

```

Advantages of static Keyword

1. **Saves Memory:** Instead of each object having its own copy of a variable, static variables are shared across all objects of the class. This saves memory because only one copy exists.
2. **Faster Access:** Since static members belong to the class itself, we don't need to create an object to use them. We can just call them using the class name, making your code run faster.

3. **Great for Utility Methods:** Some methods don't rely on object data, like `Math.sqrt()` or `Integer.parseInt()`. Marking them static allows us to use them directly without creating an object.
4. **Good for Constants:** If a value never changes, like `PI = 3.14159`, making it static final ensures that there's only one copy, making your code more efficient.
5. **Keeps Code Organized:** Static methods help keep related logic in one place. For example, if we have a `DatabaseHelper` class, its `connect()` and `disconnect()` methods can be static, so we do not have to create an object just to use them.
6. **Runs Automatically When the Class Loads:** Static blocks execute when the class is loaded. It is useful for setting up configurations like initializing a database connection or loading files before our program starts.
7. **Reduces Object Dependency:** Static methods do not rely on an object's state, making them useful in cases where creating an object would be unnecessary or inefficient.

super Keyword in Java

The **super keyword** is used to refer to the **immediate parent class object**. It is mainly used for three purposes:

1. Access parent class **data members**
2. Call parent class **methods**
3. Call parent class **constructors**

1. super with Data Member

Syntax

super.variableName;

Explanation

It is used to **access the parent class variable when the child class has the same variable name**.

Example

```
class Parent {  
    int x = 10;  
}  
  
class Child extends Parent {  
    int x = 20;  
  
    void show() {  
        System.out.println("Child x: " + x);  
        System.out.println("Parent x: " + super.x);  
    }  
}  
  
class Test {  
    public static void main(String[] args) {  
        Child obj = new Child();  
        obj.show();  
    }  
}
```

Output

Child x: 20
Parent x: 10

2. super with Method

Syntax

super.methodName();

Explanation

It is used to **call the parent class method when the child class overrides it.**

Example

```
class Parent {  
    void display() {  
        System.out.println("Parent class method");  
    }  
}  
  
class Child extends Parent {  
    void display() {  
        System.out.println("Child class method");  
    }  
  
    void show() {  
        super.display();  
    }  
}  
  
class Test {  
    public static void main(String[] args) {  
        Child obj = new Child();  
        obj.show();  
    }  
}
```

Output

Parent class method

3. super with Constructor

Syntax

super();

or

super(parameters);

Explanation

It is used to **call the constructor of the parent class**.

It must be the **first statement in the child class constructor**.

Example

```
class Parent {  
    Parent() {  
        System.out.println("Parent constructor");  
    }  
}
```

```
class Child extends Parent {  
    Child() {  
        super();  
        System.out.println("Child constructor");  
    }  
}
```

```
class Test {  
    public static void main(String[] args) {  
        Child obj = new Child();  
    }  
}
```

Output

Parent constructor
Child constructor

Types of Inheritance in Java

Inheritance allows a class to **acquire properties and methods of another class**.

Types of Inheritance

Type	Description
Single	One parent → one child
Multilevel	Parent → Child → Grandchild
Hierarchical	One parent → multiple children
Hybrid	Combination of different types

Java supports **Single, Multilevel, Hierarchical** using classes.

1. Single Inheritance

Definition: A class inherits from one parent class only.

Purpose: Promotes code reuse and avoids duplication.

```
class Animal {  
    void eat() {  
        System.out.println("Animal eats food");  
    }  
}  
  
class Dog extends Animal {  
    void bark() {  
        System.out.println("Dog barks");  
    }  
}  
  
class Test {  
    public static void main(String[] args) {  
        Dog d = new Dog();  
        d.eat();  
        d.bark();  
    }  
}
```

2. Multilevel Inheritance

- **Definition:** A class is derived from a child class, which is itself derived from a parent class.

- **Purpose:** Represents a hierarchical chain of inheritance.

```

class Animal {
    void eat() {
        System.out.println("Eating...");
    }
}

class Dog extends Animal {
    void bark() {
        System.out.println("Barking...");
    }
}

class Puppy extends Dog {
    void weep() {
        System.out.println("Weeping...");
    }
}

class Test {
    public static void main(String[] args) {
        Puppy p = new Puppy();
        p.eat();
        p.bark();
        p.weep();
    }
}

```

3. Hierarchical Inheritance

- **Definition:** Multiple classes inherit from a single parent class.
- **Purpose:** Allows different child classes to share common functionality.

```

class Animal {
    void eat() {
        System.out.println("Animal eats");
    }
}

class Dog extends Animal {
    void bark() {

```

```

        System.out.println("Dog barks");
    }
}

class Cat extends Animal {
    void meow() {
        System.out.println("Cat meows");
    }
}

class Test {
    public static void main(String[] args) {
        Dog d = new Dog();
        Cat c = new Cat();

        d.eat();
        d.bark();

        c.eat();
        c.meow();
    }
}

```

Method Overriding

Definition

Method overriding occurs when a **child class provides a specific implementation of a method that already exists in the parent class.**

Conditions

- Same **method name**
- Same **parameters**
- Same **return type**

Example

```

class Animal {
    void sound() {
        System.out.println("Animal makes sound");
    }
}

```

```

class Dog extends Animal {
    void sound() {
        System.out.println("Dog barks");
    }
}

```

```

class Test {
    public static void main(String[] args) {
        Dog d = new Dog();
        d.sound();
    }
}

```

Output

Dog barks

Method Overloading vs Method Overriding

Feature	Method Overloading	Method Overriding
Definition	Same method name with different parameters	Same method in parent and child class
Class	Same class	Parent and child class
Parameters	Must be different	Must be same
Return Type	Can be different	Must be same
Inheritance	Not required	Required
Polymorphism	Compile-time polymorphism	Runtime polymorphism

Example of Method Overloading

```

class MathOp {

    int add(int a, int b) {
        return a + b;
    }

    int add(int a, int b, int c) {
        return a + b + c;
    }
}

```

Example of Method Overriding

```
class A {  
    void show() {  
        System.out.println("Class A method");  
    }  
}  
  
class B extends A {  
    void show() {  
        System.out.println("Class B method");  
    }  
}
```

Multiple Inheritance (Not Supported in Java)

- **Definition:** A class inherits from **more than one parent class**.
- **Reason Java does NOT support it:**
 - **Diamond Problem:** If two parent classes have a method with the same signature, the compiler cannot decide which method to inherit.
 - **Example of ambiguity:**

```
class A  
{  
    void show()  
    {  
        System.out.println("A");  
    }  
}  
  
class B  
{  
    void show()  
    {  
        System.out.println("B");  
    }  
}  
  
class C extends A, B // ✗ Java does not allow this  
{  
}
```

How Java Achieves Multiple Inheritance:

- Java **supports multiple inheritance using interfaces**.
- A class can implement **multiple interfaces**, and interfaces can contain abstract methods (no code conflict).

Example Using Interfaces:

```
interface A
{
void showA();
}
interface B
{
void showB();
}
```

class C implements A, B

```
{
public void showA()
{
    System.out.println("A");
}
```

```
public void showB()
{
    System.out.println("B");
}
}
```

```
class Test {
    public static void main(String[] args) {
        C obj = new C();
        obj.showA();
        obj.showB();
    }
}
```

Output:

A
B

This is **Java's way to achieve multiple inheritance safely**.

The final Keyword in Java

In Java, the final keyword is used to **restrict the user from modifying a variable, method, or class**. It can be applied in **three main contexts**:

1. Final Variables (Data Members)

- **Definition:** A final variable **cannot be reassigned** once it has been initialized.
- **Purpose:** Used to define constants or values that should not change.

Rules:

- Must be initialized **at declaration** or in the **constructor** (for instance variables).
- Local final variables must be initialized **before use**.

Example:

```
class TestFinalVar {  
    final int MAX_VALUE = 100; // constant  
  
    final int x; // final instance variable  
  
    TestFinalVar(int val) {  
        x = val; // can initialize in constructor  
    }  
  
    void show() {  
        System.out.println("MAX_VALUE: " + MAX_VALUE);  
        System.out.println("x: " + x);  
    }  
  
    public static void main(String[] args) {  
        TestFinalVar obj = new TestFinalVar(50);  
        obj.show();  
        // obj.MAX_VALUE = 200; // ✗ Error: cannot assign a value to final variable  
    }  
}
```

Output:

MAX_VALUE: 100
x: 50

- ☑ final ensures the value remains constant after initialization.

2. Final Methods

- **Definition:** A final method **cannot be overridden** by subclasses.
- **Purpose:** Protects the implementation of the method.

Example:

```
class Parent {  
    final void display() {  
        System.out.println("This is a final method");  
    }  
}  
  
class Child extends Parent {  
    void display() //Error- display( ) method cannot be overridden by Child class.  
    {  
        System.out.println("Trying to override");  
    }  
}  
  
class TestFinalMethod {  
    public static void main(String[] args) {  
        Child c = new Child();  
        c.display();  
    }  
}
```

Output:

This is a final method

- ☑ final prevents subclass modification, ensuring consistent behavior.

3. Final Classes

- **Definition:** A final class **cannot be extended**.
- **Purpose:** Prevents inheritance for security or design reasons.

Example:

```

final class FinalClass {
    void show() {
        System.out.println("This is a final class");
    }
}

```

```

// class Child extends FinalClass // ✗ Error: Cannot inherit from final class
{
}

```

```

class TestFinalClass {
    public static void main(String[] args) {
        FinalClass obj = new FinalClass();
        obj.show();
    }
}

```

Output:

This is a final class

☒ final classes are often used for utility or immutable classes (e.g., java.lang.String).

Summary Table of final Keyword

Usage	Effect	Example
Final Variable	Value cannot change after initialization	final int x = 10;
Final Method	Cannot be overridden in subclasses	final void show() {}
Final Class	Cannot be extended	final class MyClass {}

- final + static variables are **constants** by convention:

final static double PI = 3.14159;

Java throw and throws Keywords

throw Keyword

Purpose: Used to explicitly throw an exception from a method or block of code.

Syntax:

throw new ExceptionType("message");

- **Definition:** Used to **explicitly throw an exception** from a method or block.
- **Purpose:** To signal that an **exception has occurred** at a specific point in the program.
- **Can throw:** Only **one exception** at a time.
- **Type:** Used inside a method or a block.

Example 1:

```
public class ThrowExample1 {  
    public static void main(String[] args) {  
        int age = 15;  
        if (age < 18) {  
            throw new ArithmeticException("Not eligible to vote");  
        }  
        System.out.println("You can vote");  
    }  
}
```

Example 2:

```
public class ThrowExample2 {  
    static void checkNumber(int num) {  
        if (num < 0) {
```

```

        throw new IllegalArgumentException("Number cannot be
negative");
    }
    System.out.println("Valid number");
}

public static void main(String[] args) {
    checkNumber(-5);
}
}

```

throws Keyword

Purpose: Used in method declaration to indicate that the method may throw exceptions.

Syntax:

returnType methodName() throws ExceptionType1, ExceptionType2 { }

- **Definition:** Used in a **method declaration** to **declare the exceptions that a method might throw**.
- **Purpose:** To inform the **caller of the method** that it should handle the exception.
- **Can declare: Multiple exceptions** separated by commas.
- **Type:** Part of **method signature**.

Example 1:

```

import java.io.*;

public class ThrowsExample1 {
    public static void readFile() throws IOException {
        FileReader file = new FileReader("data.txt");
        BufferedReader br = new BufferedReader(file);
        System.out.println(br.readLine());
    }
}

```

```

    }

    public static void main(String[] args) {
        try {
            readFile();
        } catch (IOException e) {
            System.out.println("File not found");
        }
    }
}

```

Example 2: Multiple exceptions

```

public class ThrowsExample2 {
    static void test(int a) throws ArithmeticException, NullPointerException {
        if (a == 0) throw new ArithmeticException("Divide by zero");
        if (a < 0) throw new NullPointerException("Negative value");
        System.out.println("Valid input");
    }

    public static void main(String[] args) {
        try {
            test(0);
        } catch (Exception e) {
            System.out.println(e);
        }
    }
}

```

User-Defined (Custom) Exception

A user-defined exception is created by extending the Exception class.

Example:

```

class InvalidAgeException extends Exception {
    public InvalidAgeException(String message) {
        super(message);
    }
}

```

```
}  
}
```

```
public class CustomExceptionExample {  
    static void checkAge(int age) throws InvalidAgeException {  
        if (age < 18) {  
            throw new InvalidAgeException("Not eligible");  
        }  
        System.out.println("Eligible to vote");  
    }  
}
```

```
    public static void main(String[] args) {  
        try {  
            checkAge(16);  
        } catch (InvalidAgeException e) {  
            System.out.println(e.getMessage());  
        }  
    }  
}
```

Key Differences Between throw and throws

Feature	throw	throws
Purpose	To throw an exception explicitly	To declare exceptions a method may throw
Usage	Inside method/block	In method declaration
Number of exceptions	Only one exception at a time	Can declare multiple exceptions separated by commas
Syntax	throw new ExceptionType("msg");	void method() throws ExceptionType1,

Feature	throw	throws
		ExceptionType2
Runtime/Compile-time	Used at runtime	Checked at compile-time for checked exceptions
Example	throw new IOException("File error");	void readFile() throws IOException